

EECS 836: Machine Learning

Zijun Yao

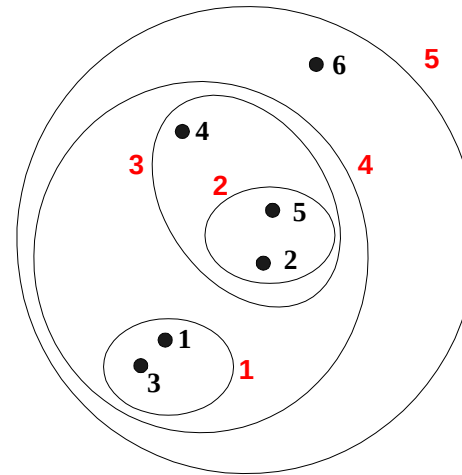
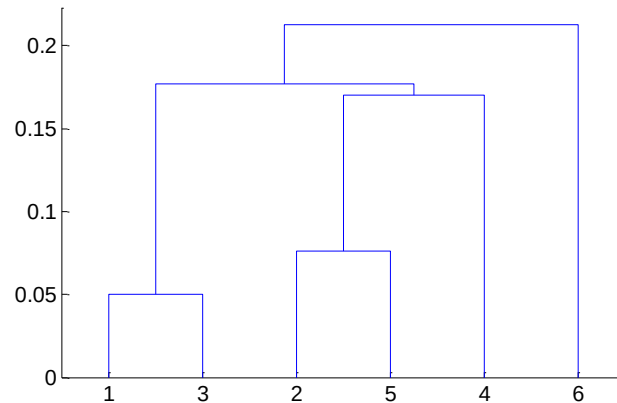
Assistant Professor, EECS Department
The University of Kansas

Agenda

- Hierarchical Clustering
 - Different Types of Inter-Cluster Distance
 - Complexity and Limitations
- Recommender Systems
 - Content Filtering
 - Collaborative Filtering

Hierarchical clustering

- Produces a set of nested clusters organized as a hierarchical tree
- Can be visualized as a **dendrogram**
 - A tree like diagram that records the sequences of merges or splits
 - X-axis: data points, Y-axis: distance between clusters



Strengths of hierarchical clustering

- Do not have to assume any particular number of clusters
 - Any desired number of clusters can be obtained by 'cutting' the dendrogram at the proper level
- **Dendrograms** may correspond to meaningful **taxonomies**
 - Example in biological sciences (e.g., animal kingdom, phylogeny reconstruction, ...)

Hierarchical clustering

- Two main types of hierarchical clustering
 - Agglomerative:
 - Start with the points as **individual clusters**
 - At each step, merge the closest pair of clusters until only one cluster (or k clusters) left
 - Divisive:
 - Start with one, **all-inclusive cluster**
 - At each step, split a cluster until each cluster contains an individual point (or there are k clusters)
- Traditional hierarchical algorithms use a similarity or distance matrix
 - Merge or split one cluster at a time

Agglomerative clustering algorithm

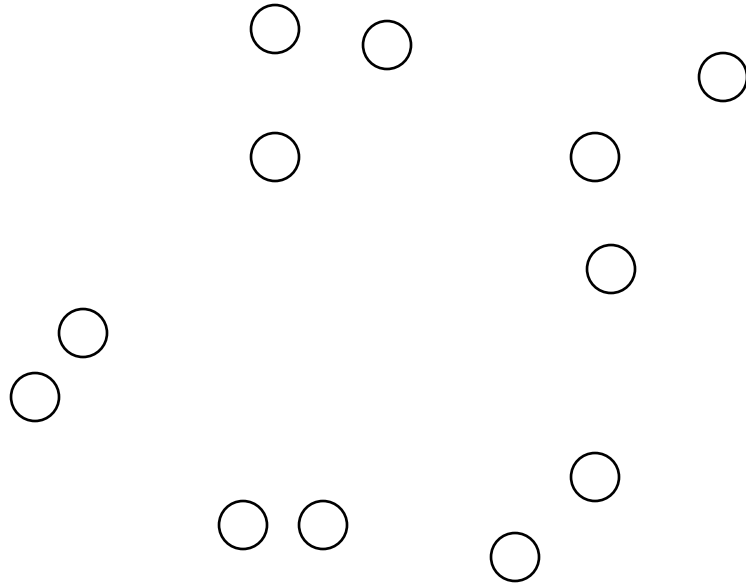
- **Key Idea: Successively merge closest clusters**
- Basic algorithm
 1. Compute the **proximity matrix**
 2. Let each data point be a cluster
 3. **Repeat**
 4. **Merge** the two closest clusters
 5. **Update** the proximity matrix
 6. **Until** only a single cluster remains
- Key operation is the computation of the **proximity of two clusters**
 - Different approaches to defining the distance between clusters distinguish the different algorithms

	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

Steps 1 and 2

- Start with clusters of individual points and a proximity matrix



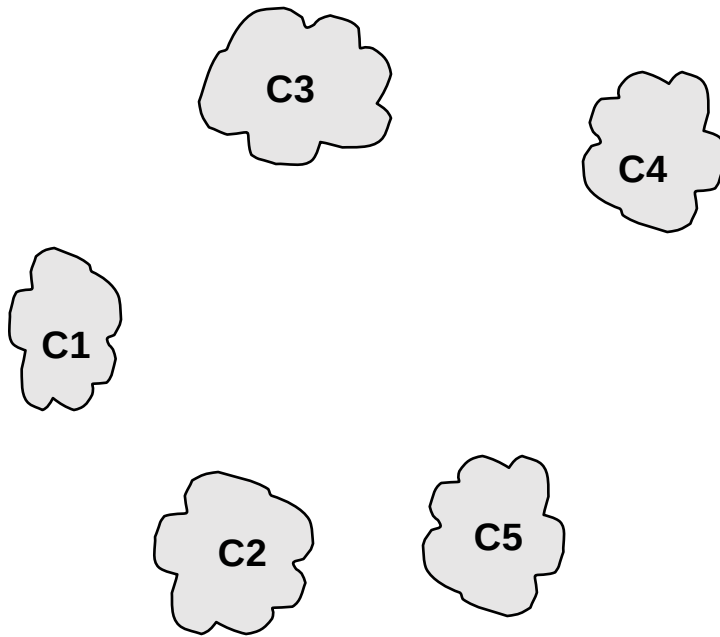
	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix



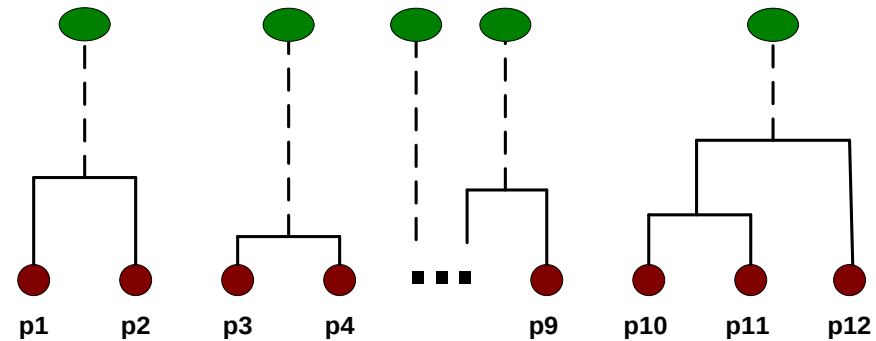
Intermediate Situation

- After some merging steps, we have some clusters



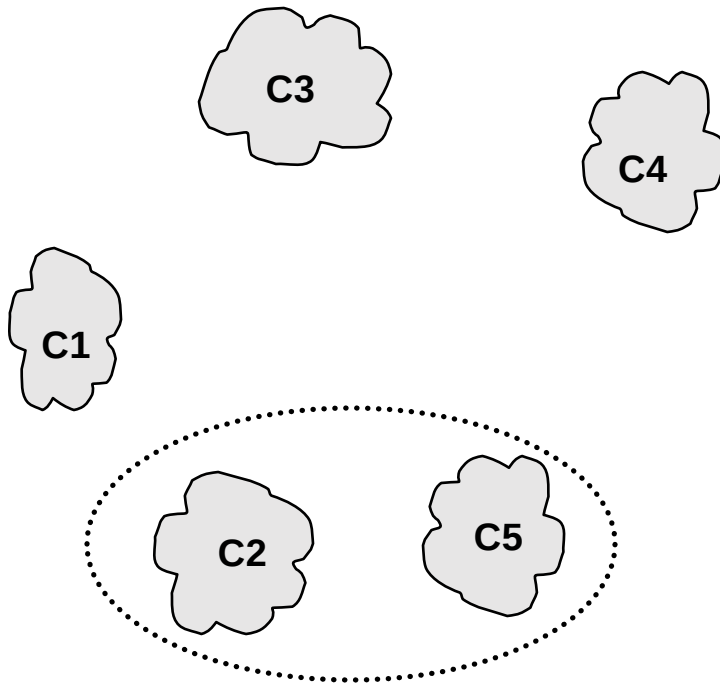
	C1	C2	C3	C4	C5
C1					
C2					
C3					
C4					
C5					

Proximity Matrix



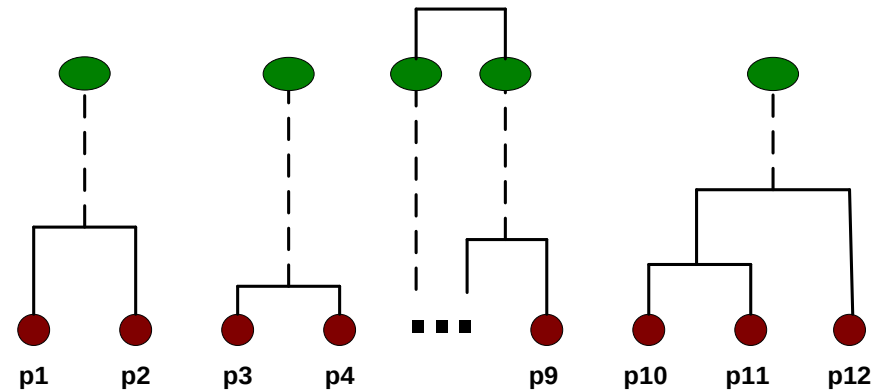
Step 4

- We want to merge the two closest clusters (C2 and C5) and update the proximity matrix.



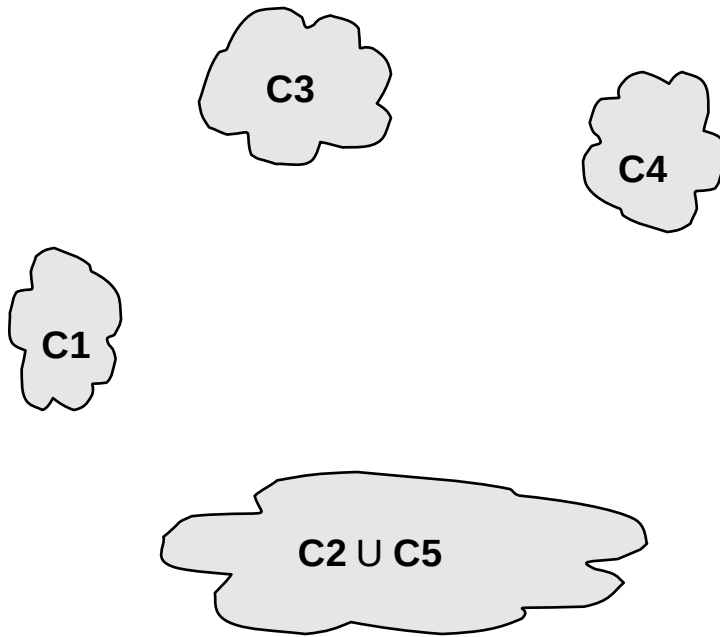
	C1	C2	C3	C4	C5
C1					
C2					
C3					
C4					
C5					

Proximity Matrix



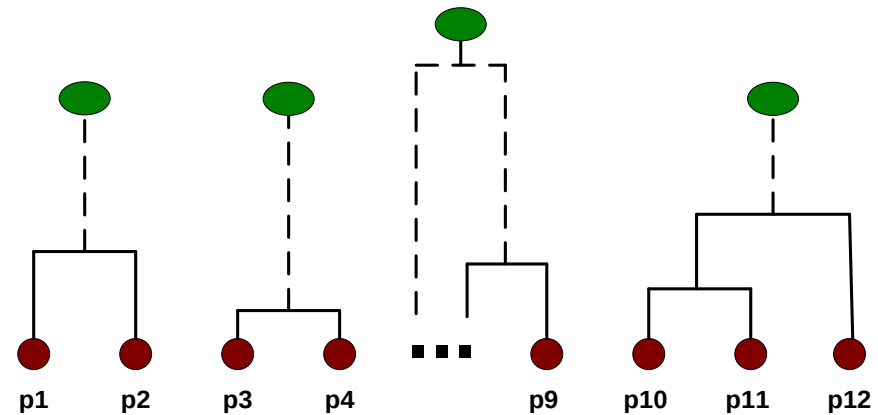
Step 5

- The question is “How do we update the proximity matrix?”

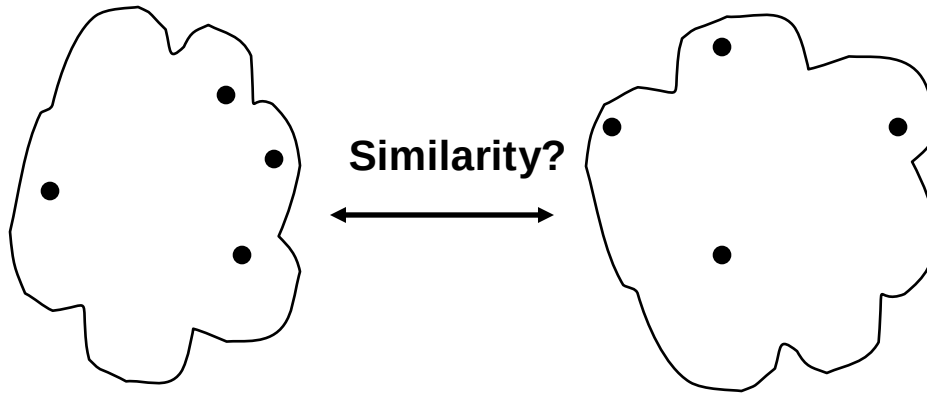


	C1	C2 U C5	C3	C4
C1		?		
C2 U C5	?	?	?	?
C3		?		
C4		?		

Proximity Matrix



How to define inter-cluster distance

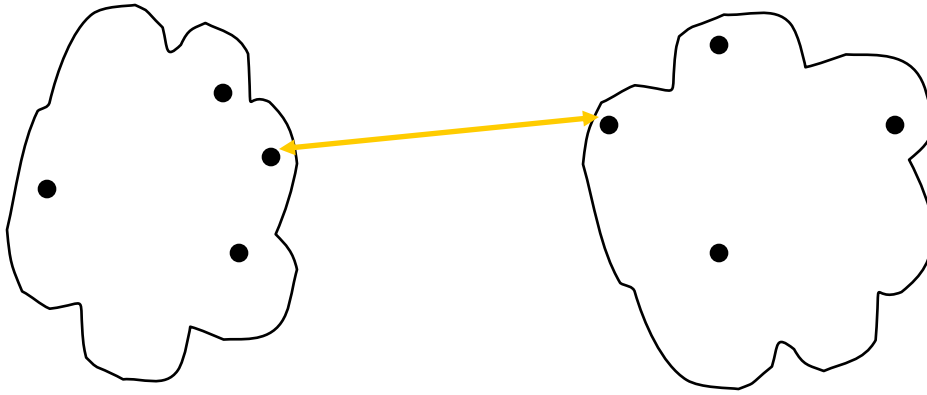


- MIN
- MAX
- Group Average
- Distance Between Centroids
- Other methods driven by an objective function
 - Ward's Method uses squared error

	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						
.						

• **Proximity Matrix**

How to Define Inter-Cluster Similarity

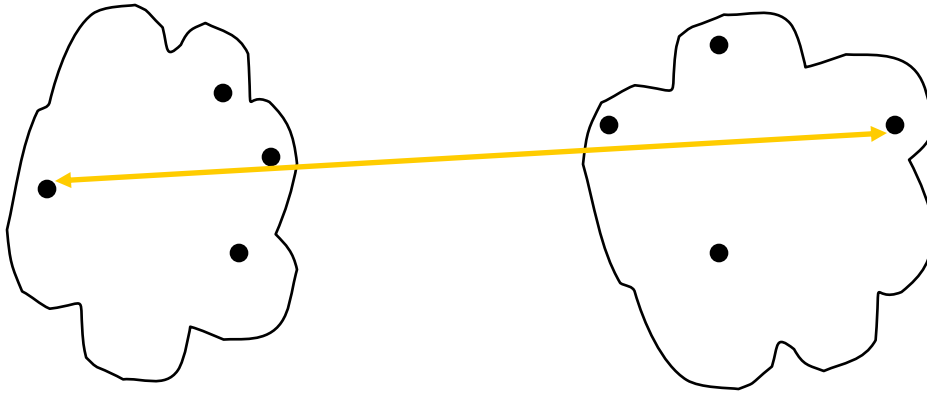


- **MIN**
- **MAX**
- Group Average
- Distance Between Centroids
- Other methods driven by an objective function
 - Ward's Method uses squared error

	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						

• **Proximity Matrix**

How to Define Inter-Cluster Similarity

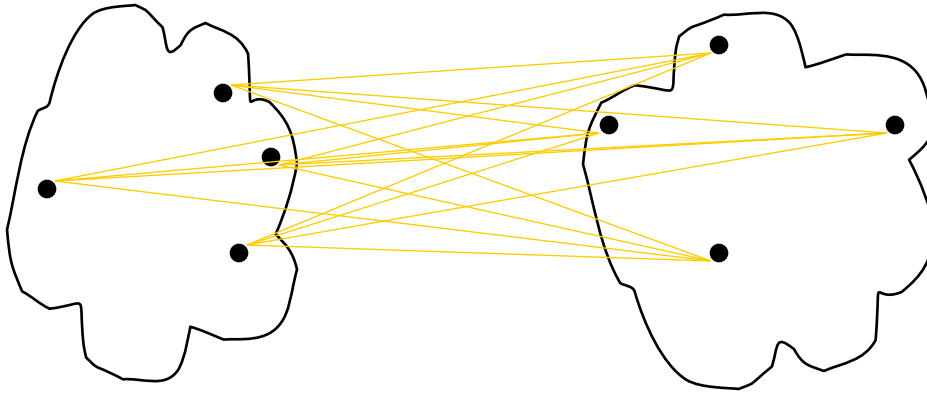


- MIN
- **MAX**
- Group Average
- Distance Between Centroids
- Other methods driven by an objective function
 - Ward's Method uses squared error

	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						
.						

• **Proximity Matrix**

How to Define Inter-Cluster Similarity

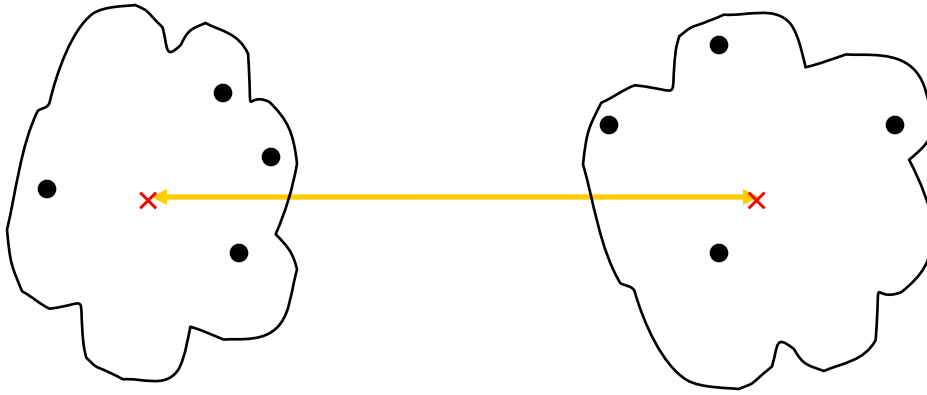


- MIN
- MAX
- **Group Average**
- Distance Between Centroids
- Other methods driven by an objective function
 - Ward's Method uses squared error

	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						

• **Proximity Matrix**

How to Define Inter-Cluster Similarity



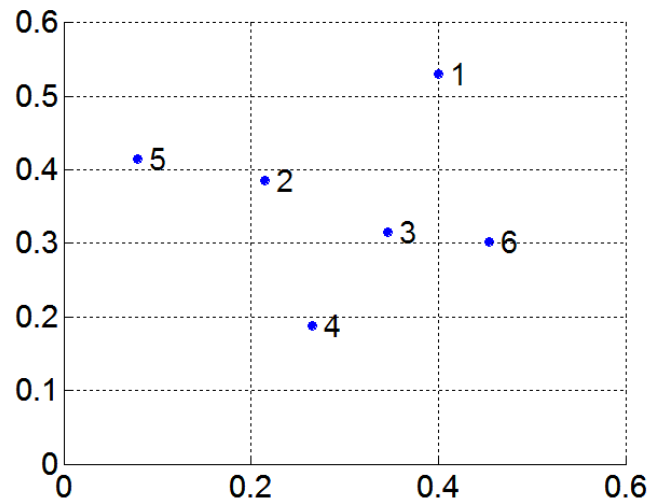
- MIN
- MAX
- Group Average
- Distance Between Centroids
- Other methods driven by an objective function
 - Ward's Method uses squared error

	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						

• Proximity Matrix

MIN or single link

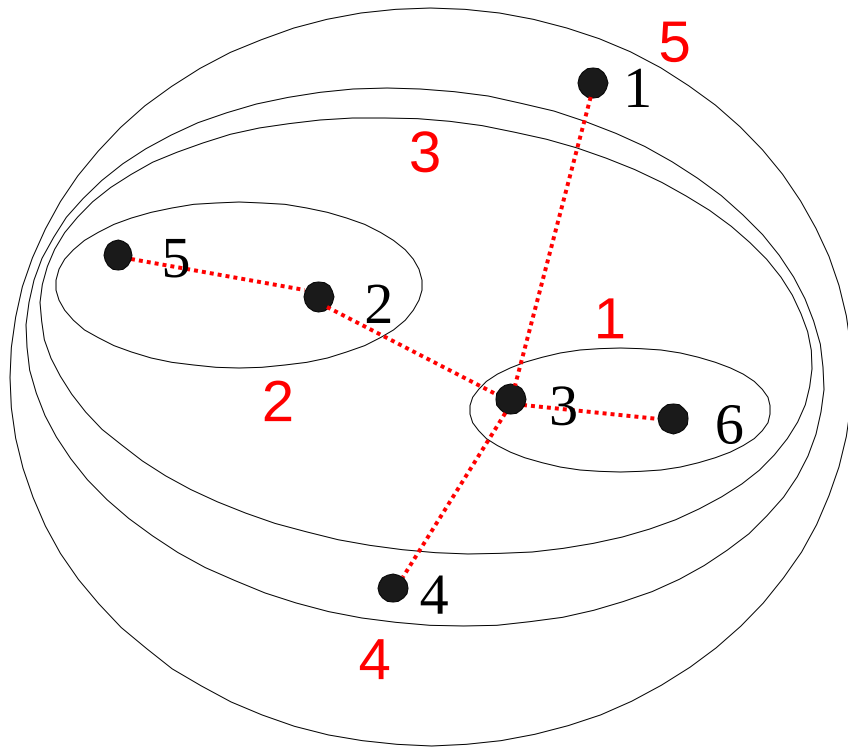
- Proximity of two clusters is based on the **two closest points** in the different clusters
 - Determined by one pair of points
 - One single link can decide a merge
- Example:



Distance Matrix:

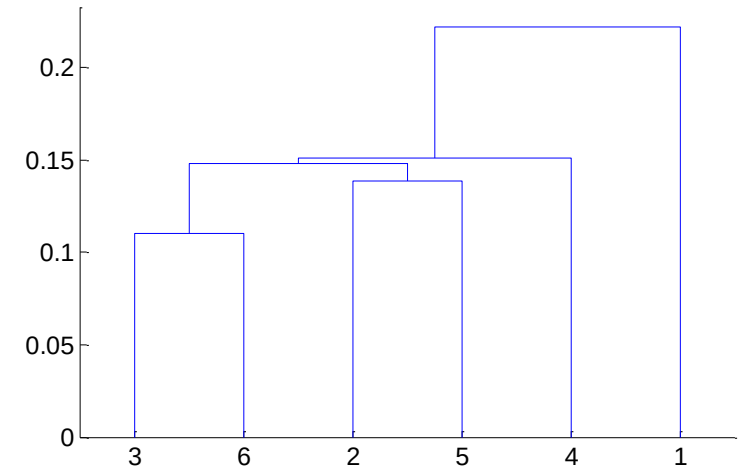
	1	2	3	4	5	6
1	0	.24	.22	.37	.34	.23
2	.24	0	.15	.20	.14	.25
3	.22	.15	0	.15	.28	.11
4	.37	.20	.15	0	.29	.22
5	.34	.14	.28	.29	0	.39
6	.23	.25	.11	.22	.39	0

Hierarchical clustering: MIN



Nested Clusters

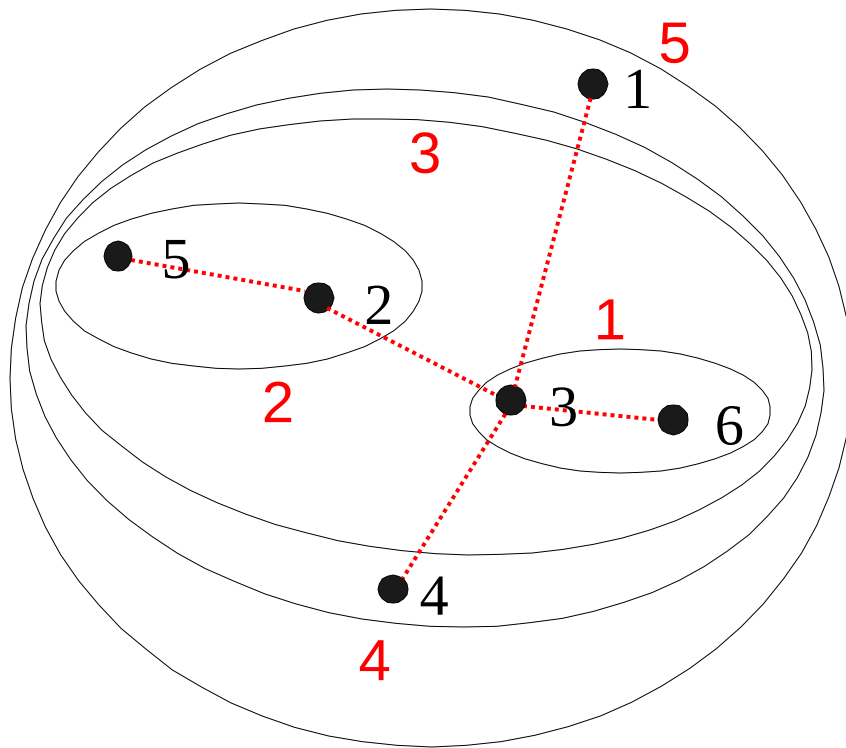
	1	2	3	4	5	6
1	0	.24	.22	.37	.34	.23
2	.24	0	.15	.20	.14	.25
3	.22	.15	0	.15	.28	.11
4	.37	.20	.15	0	.29	.22
5	.34	.14	.28	.29	0	.39
6	.23	.25	.11	.22	.39	0



Dendrogram

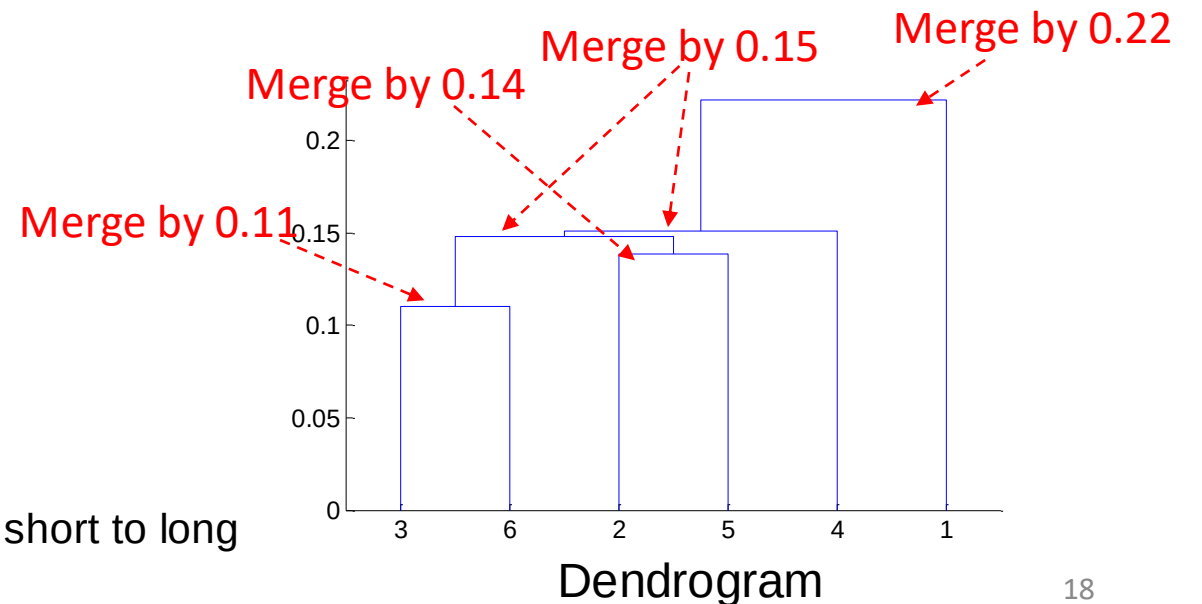
- Linkage between any two points are considered in orders from short to long
- Red: shortest edges that achieve single linkage of two clusters

Hierarchical clustering: MIN



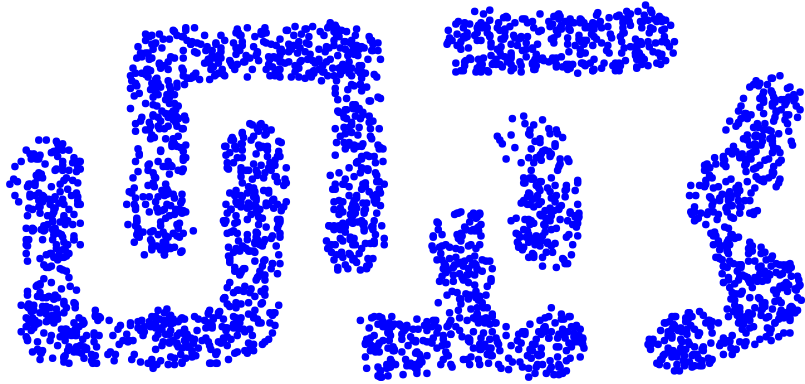
Nested Clusters

	1	2	3	4	5	6
1	0	.24	.22	.37	.34	.23
2	.24	0	.15	.20	.14	.25
3	.22	.15	0	.15	.28	.11
4	.37	.20	.15	0	.29	.22
5	.34	.14	.28	.29	0	.39
6	.23	.25	.11	.22	.39	0

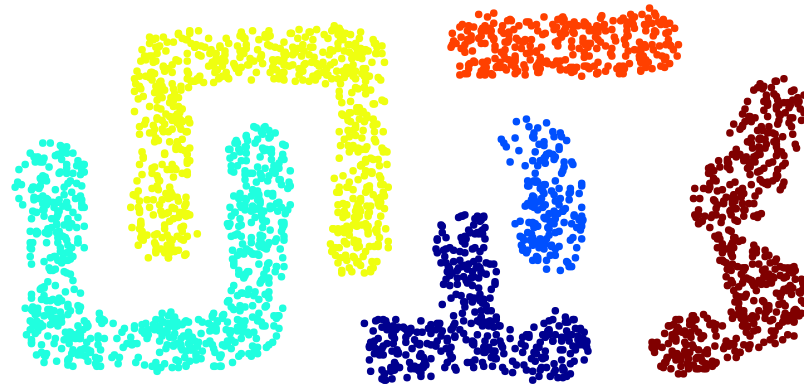


- Linkage between any two points are considered in orders from short to long
- Red: shortest edges that achieve single linkage of two clusters

Strength of MIN



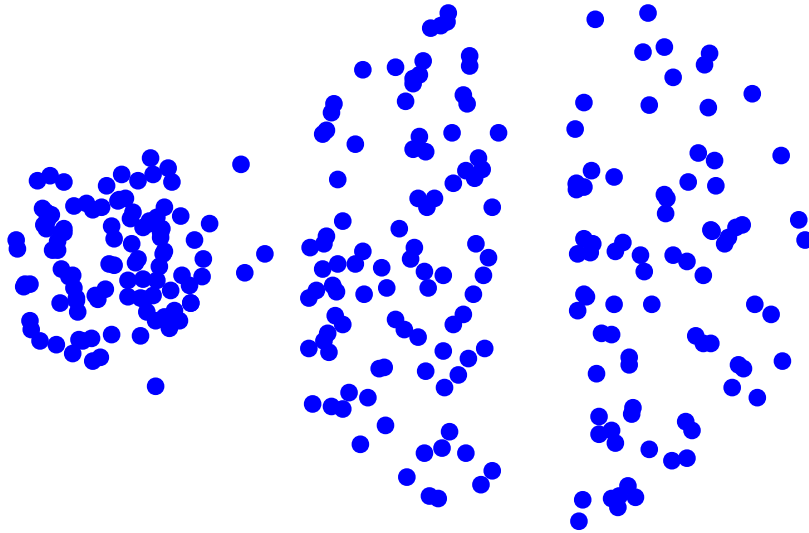
Original Points



Six Clusters

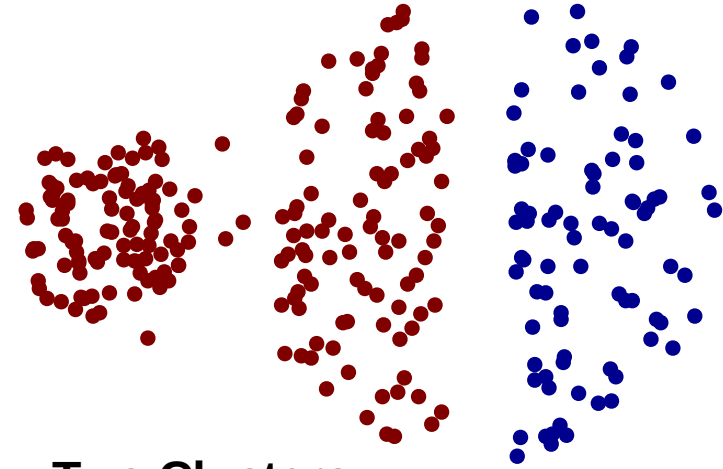
- Can handle non-elliptical shapes

Limitations of MIN

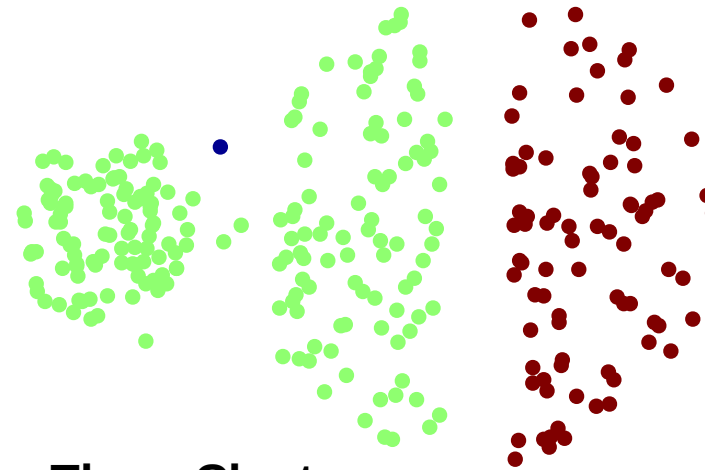


Original Points

- Sensitive to noise



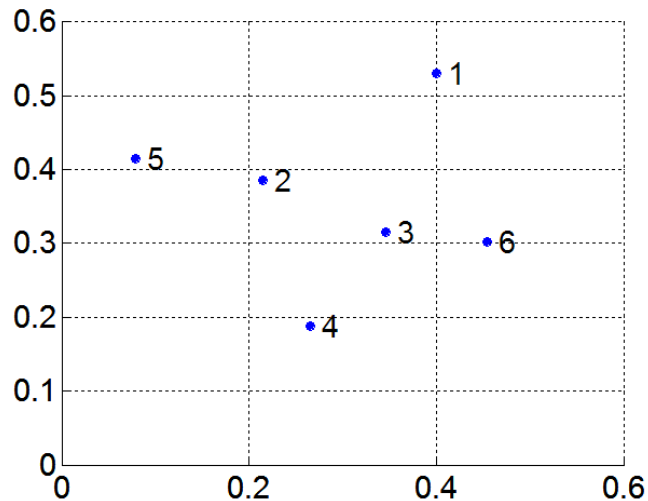
Two Clusters



Three Clusters

MAX or Complete Linkage

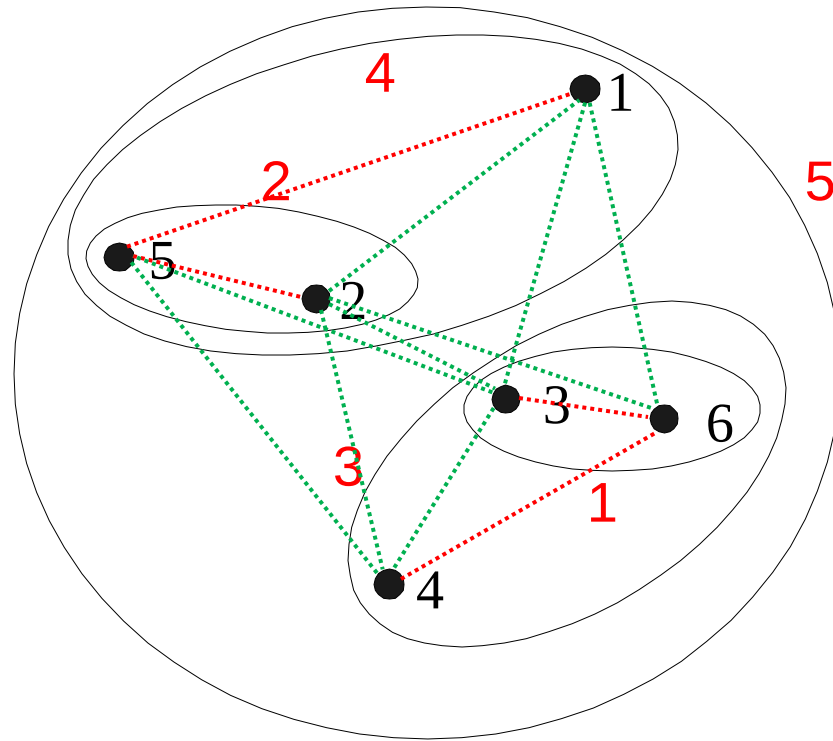
- Proximity of two clusters is based on the **two most distant points** in the different clusters
 - Determined by all pairs of points in the two clusters



Distance Matrix:

	1	2	3	4	5	6
1	0	.24	.22	.37	.34	.23
2	.24	0	.15	.20	.14	.25
3	.22	.15	0	.15	.28	.11
4	.37	.20	.15	0	.29	.22
5	.34	.14	.28	.29	0	.39
6	.23	.25	.11	.22	.39	0

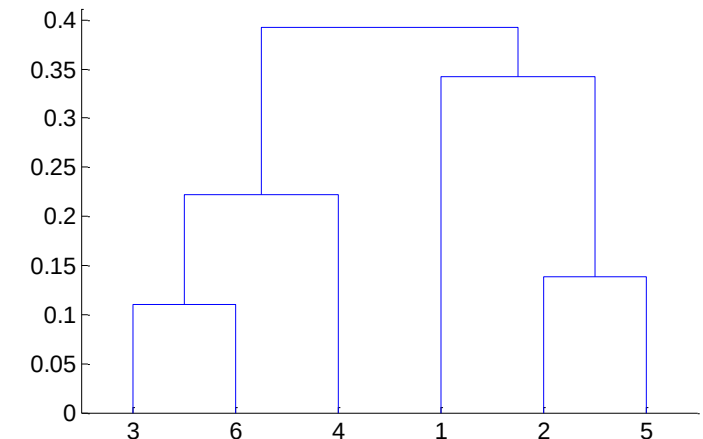
Hierarchical clustering: MAX



Nested Clusters

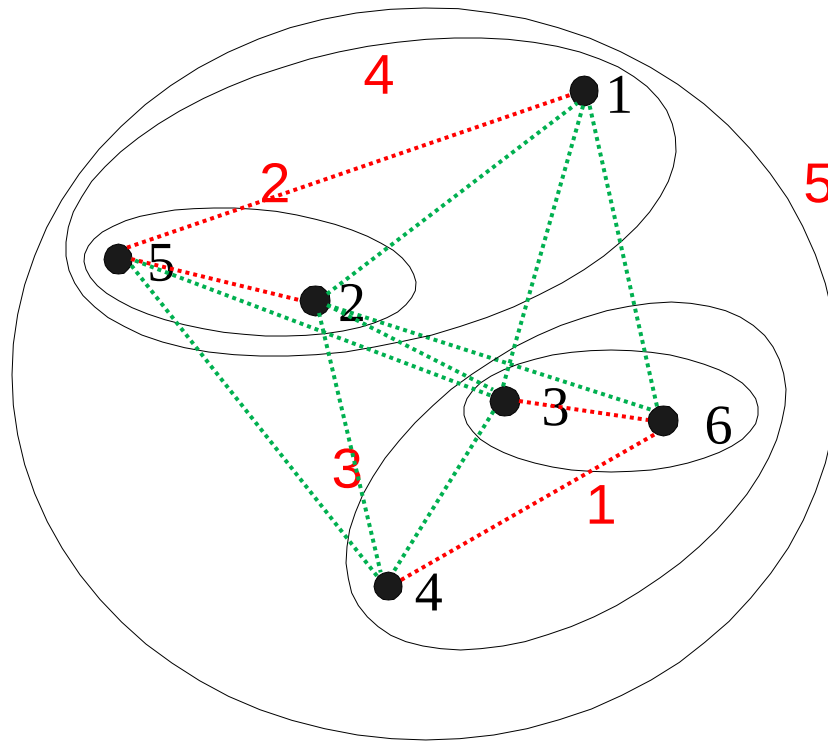
- Linkage between any two points are considered in orders from short to long
- Green: edges that are considered, but can not decide the longest distance
- Red: longest edges that achieve complete linkage of two clusters

	1	2	3	4	5	6
1	0	.24	.22	.37	.34	.23
2	.24	0	.15	.20	.14	.25
3	.22	.15	0	.15	.28	.11
4	.37	.20	.15	0	.29	.22
5	.34	.14	.28	.29	0	.39
6	.23	.25	.11	.22	.39	0



Dendrogram

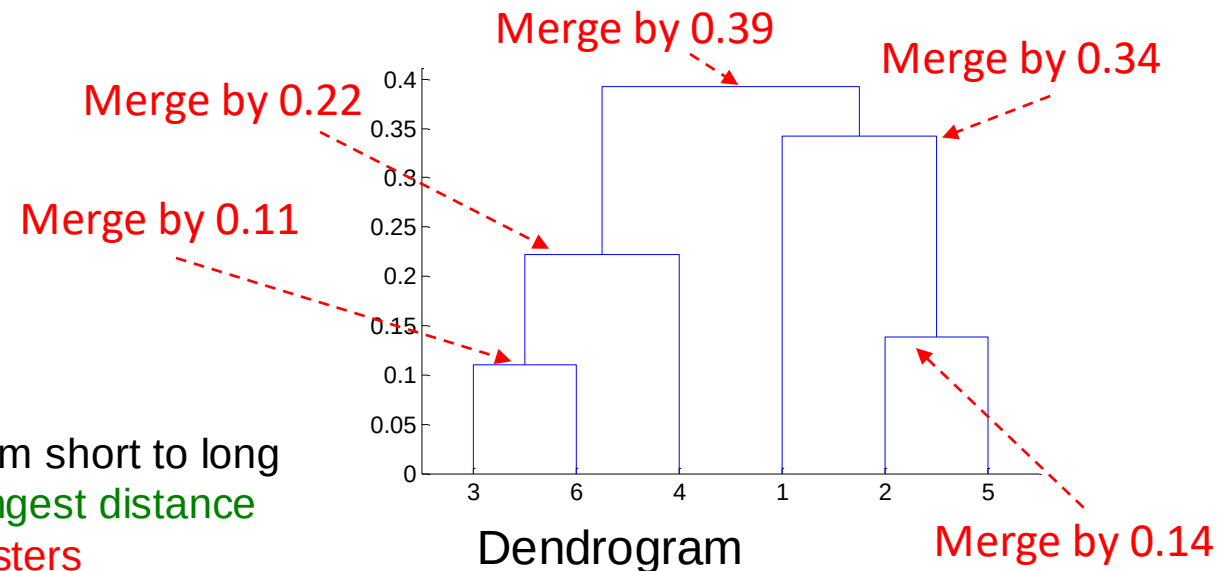
Hierarchical clustering: MAX



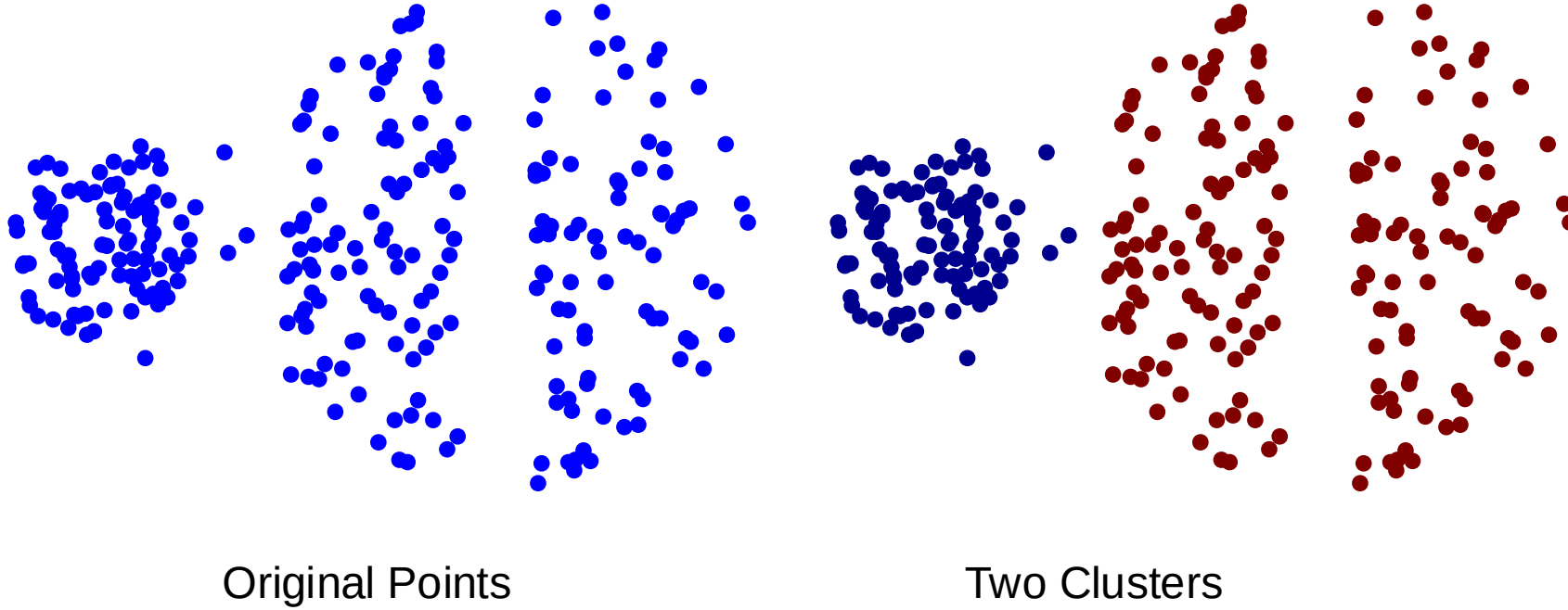
Nested Clusters

- Linkage between any two points are considered in orders from short to long
- Green: edges that are considered, but can not decide the longest distance
- Red: longest edges that achieve complete linkage of two clusters

	1	2	3	4	5	6
1	0	.24	.22	.37	.34	.23
2	.24	0	.15	.20	.14	.25
3	.22	.15	0	.15	.28	.11
4	.37	.20	.15	0	.29	.22
5	.34	.14	.28	.29	0	.39
6	.23	.25	.11	.22	.39	0

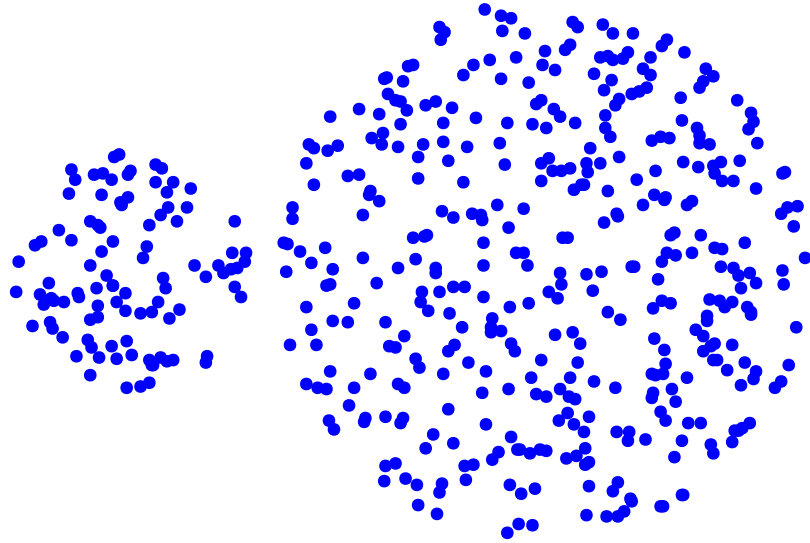


Strength of MAX

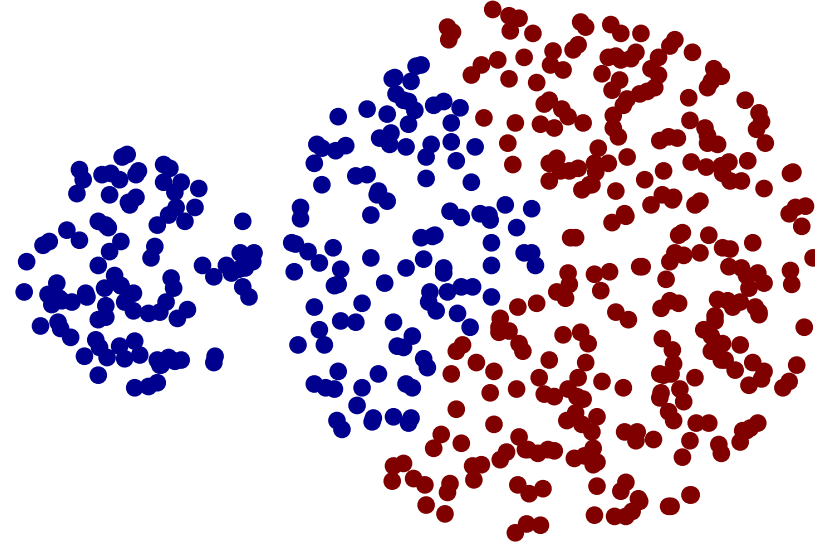


- Less susceptible to noise and outliers

Limitations of MAX



Original Points



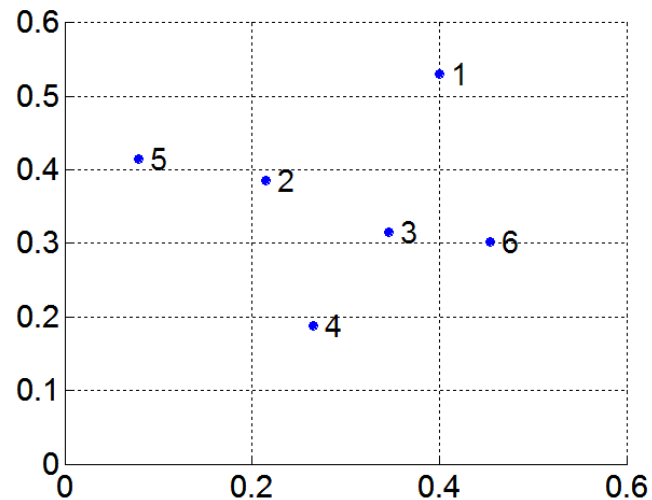
Two Clusters

- Tends to break large clusters
- Biased towards globular clusters

Group average

- Proximity of two clusters is the average of pairwise proximity between points in the two clusters.

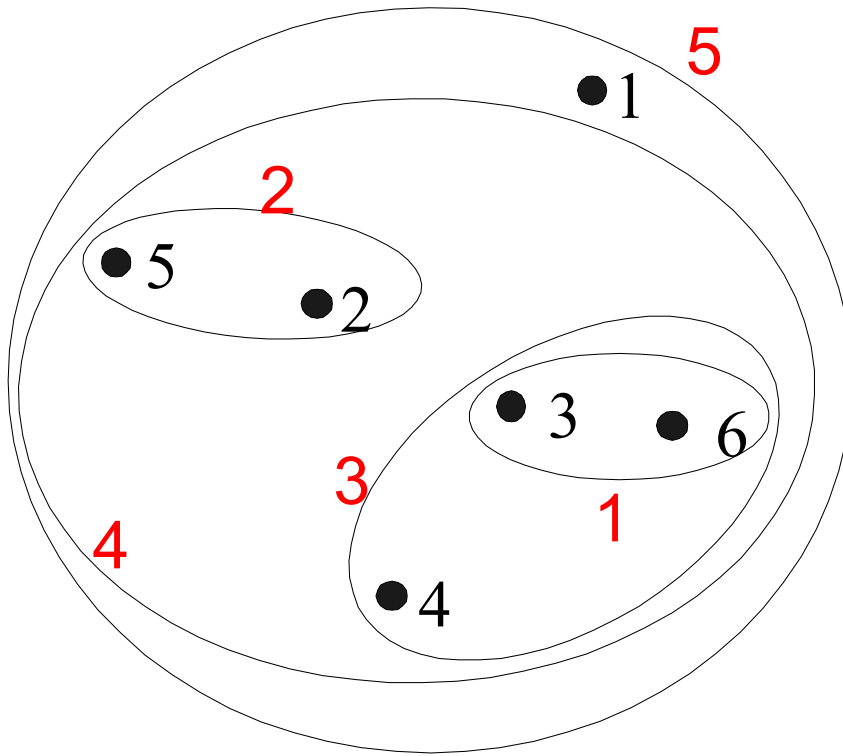
$$\text{proximity}(\text{cluster}_i, \text{cluster}_j) = \frac{\sum_{p_i \in \text{cluster}_i, p_j \in \text{cluster}_j} \text{dist}(p_i, p_j)}{|\text{cluster}_i| \times |\text{cluster}_j|}$$



Distance Matrix:

	1	2	3	4	5	6
1	0	.24	.22	.37	.34	.23
2	.24	0	.15	.20	.14	.25
3	.22	.15	0	.15	.28	.11
4	.37	.20	.15	0	.29	.22
5	.34	.14	.28	.29	0	.39
6	.23	.25	.11	.22	.39	0

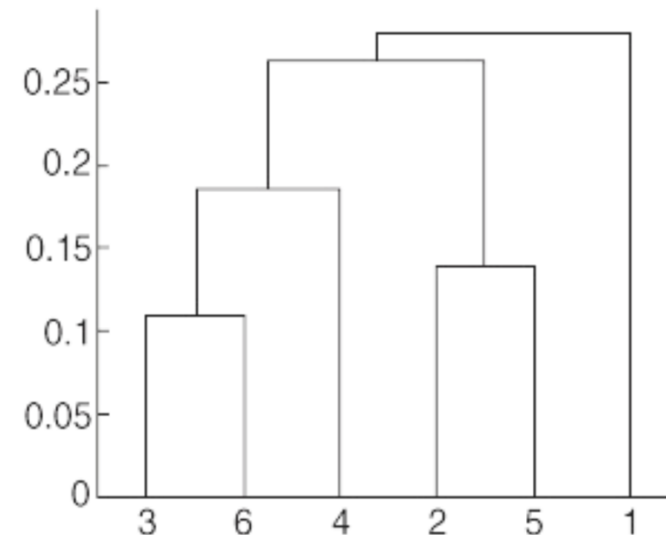
Hierarchical clustering: group average



Nested Clusters

	1	2	3	4	5	6
1	0	.24	.22	.37	.34	.23
2	.24	0	.15	.20	.14	.25
3	.22	.15	0	.15	.28	.11
4	.37	.20	.15	0	.29	.22
5	.34	.14	.28	.29	0	.39
6	.23	.25	.11	.22	.39	0

Dendrogram



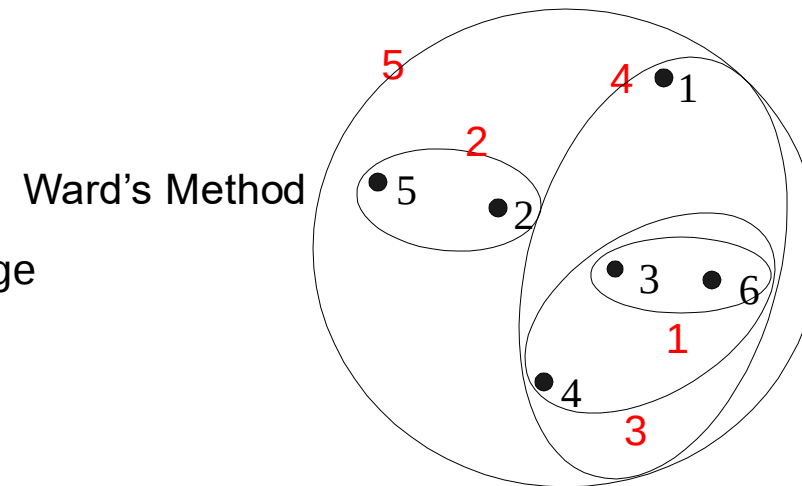
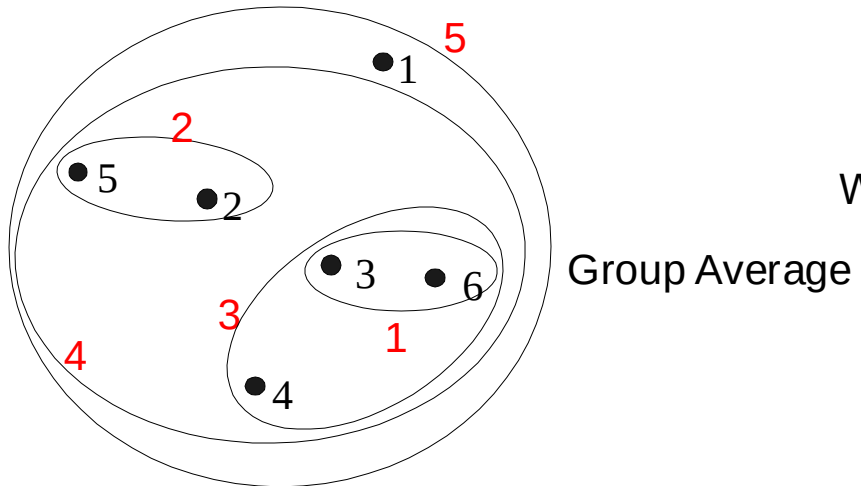
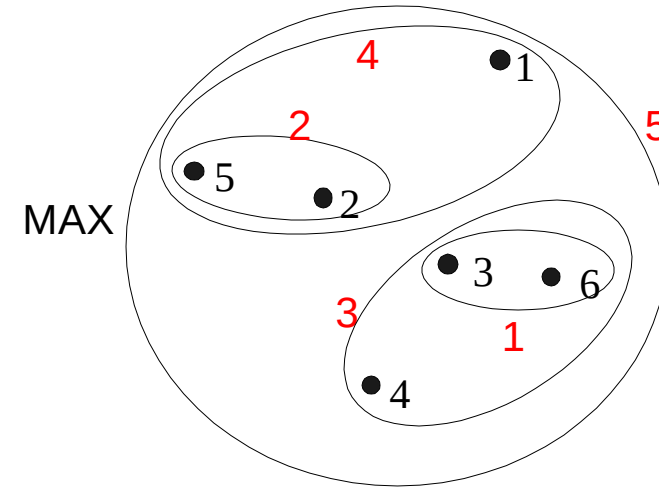
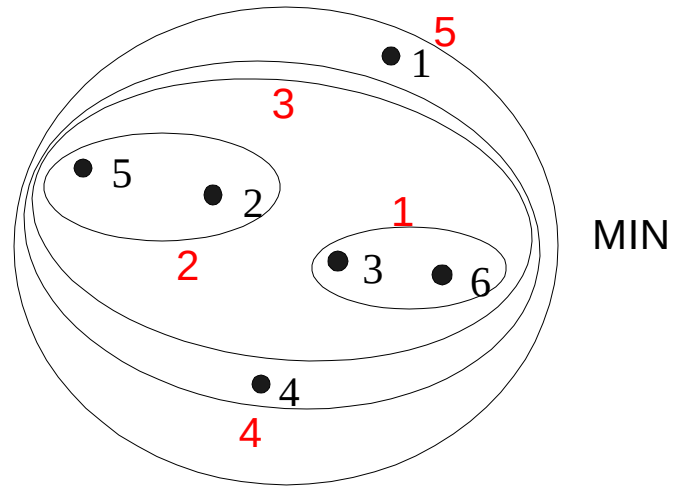
Hierarchical clustering: group average

- Compromise between Single and Complete Link
- Strengths
 - Less susceptible to noise
- Limitations
 - Biased towards globular clusters

Cluster similarity: Ward's method

- Similarity of two clusters is based on the **increase** in **squared error (SSE)** when two clusters are merged
 - Similar to group average if distance between points is distance squared
- Less susceptible to noise and outliers
- Biased towards globular clusters
- Hierarchical analogue of K-means
 - Can be used to initialize K-means

Hierarchical clustering: comparison



Hierarchical clustering: time and space requirements

- $O(N^2)$ space since it uses the proximity matrix.
 - N is the number of points.
- $O(N^3)$ time in many cases
 - There are N steps and at each step the size, N^2 , proximity matrix must be updated and searched
 - Complexity can be reduced to $O(N^2 \log(N))$ time with some cleverness

Hierarchical clustering: limitations

- Computational complexity in time and space
- Once a decision is made to combine two clusters, it cannot be undone
- No global objective function is directly minimized
- Different schemes have problems with one or more of the following:
 - Sensitivity to noise
 - Difficulty handling clusters of different sizes and non-globular shapes
 - Breaking large clusters

Agenda

- Hierarchical Clustering
 - Different Types of Inter-Cluster Distance
 - Complexity and Limitations
- Recommender Systems
 - Content Filtering
 - Collaborative Filtering

Personalized recommendation

- Recommendation systems
 - When a user rates a **movie**, we want to recommend movies that the user may like
 - When a user likes a **song**, we want to recommend other songs that they may like



The image shows the Netflix Prize leaderboard page. A large yellow banner at the top says "COMPLETED". Below it, the "Leaderboard" section is titled "Problem Setup".

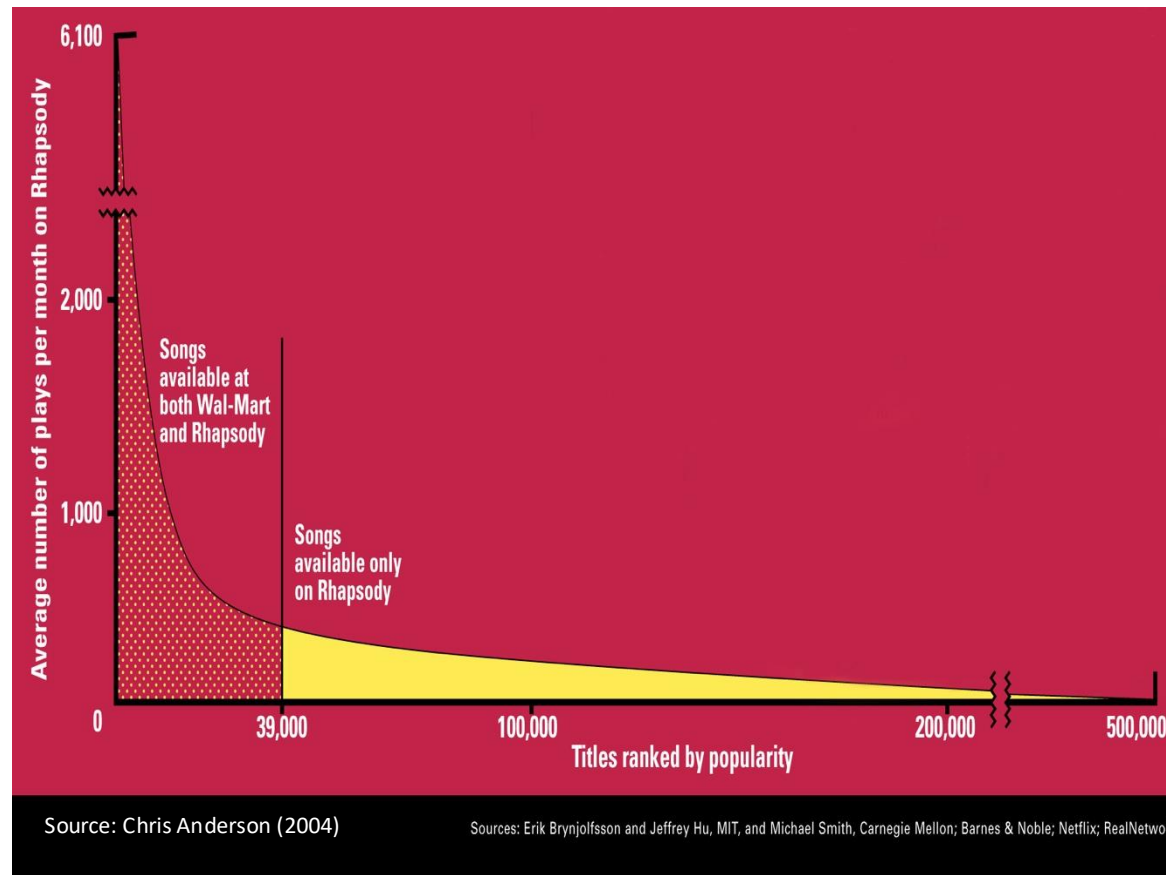
Problem Setup

- 500,000 users
- 20,000 movies
- 100 million ratings
- Goal: To obtain lower root mean squared error (RMSE) than Netflix's existing system on 3 million held out ratings

Rank	Team Name	Best Test Score	% Improvement	Best Submit Time
9	Feedsz	0.8622	9.46	2009-07-12 13:11:51
10	BigChaos	0.8623	9.47	2009-04-07 12:33:59
11	Opera Solutions	0.8623	9.47	2009-07-24 00:34:07
12	BellKor	0.8624	9.46	2009-07-26 17:19:11

The long tail effect

- A big success by exploiting **long tail**
 - A small number of popular items and the rest not sell that well
 - The possibility to automatically explore and discover



Interaction (preference) matrix

	Harry Potter 1	Harry Potter 2	Harry Potter 3	Twilight	Star Wars 1	Star Wars 2	Star Wars 3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

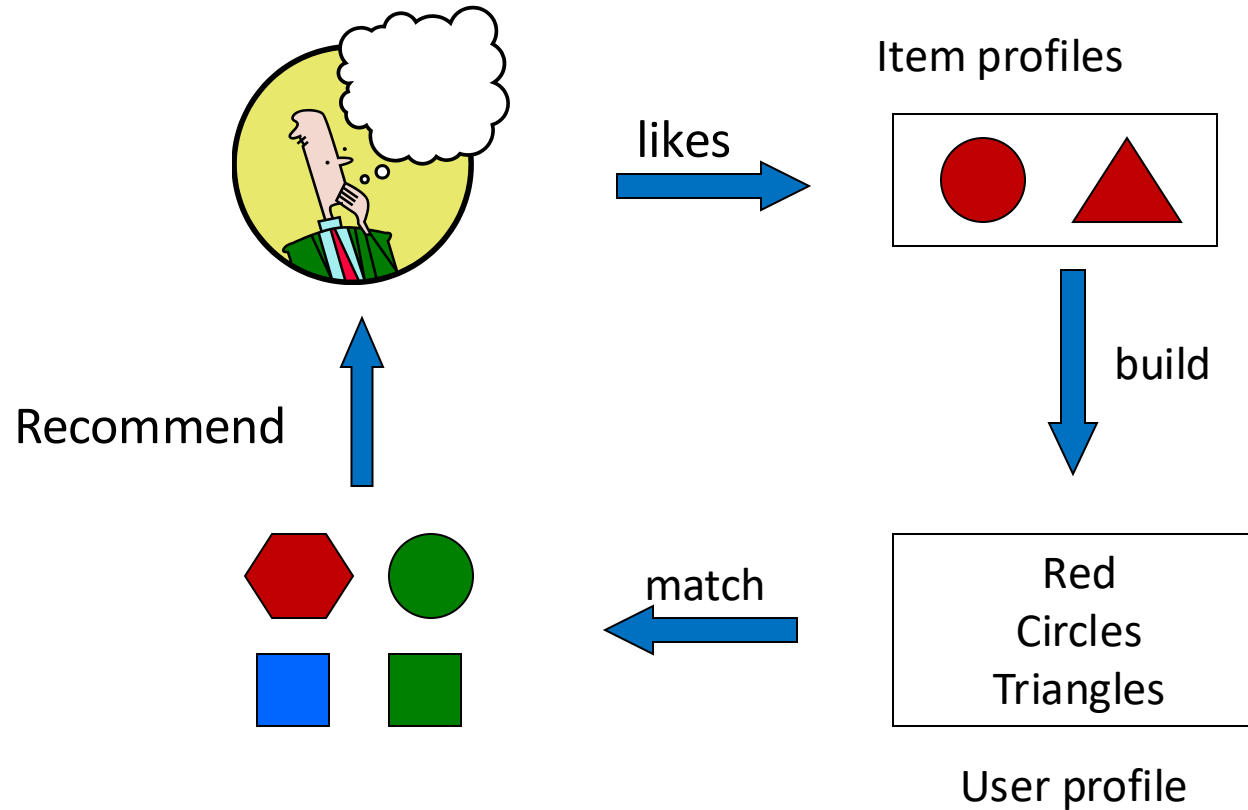
Rows: Users

Columns: Movies (or other items)

Values: The rating of the user for the movie

How can we fill the empty entries of the matrix?

Intuition of recommendation



Two Types of Recommender Systems

Content Filtering

- *Example:* Pandora.com music recommendations (Music Genome Project)
- **Con:** Assumes access to side information about items (e.g. properties of a song)
- **Pro:** Got a new item to add? No problem, just be sure to include the side information

Collaborative Filtering

- *Example:* Netflix movie recommendations
- **Pro:** Does not assume access to side information about items (e.g. does not need to know about movie genres)
- **Con:** Does not work on new items that have no ratings

Content-based recommendation

	Harry Potter 1	Harry Potter 2	Harry Potter 3	Twilight	Star Wars 1	Star Wars 2	Star Wars 3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

- Someone who likes one of the Harry Potter (or Star Wars)
- movies is likely to like the rest
 - Same actors, similar story, same genre

Approach

- Map items into a **feature space**:
 - For movies:
 - Actors, directors, genre, rating, year,...
 - Challenge: make all features compatible.
 - For documents?
- To compare items with users we need to **map** users to the same feature space.
 - Take all the movies that the user has seen and take the average vector, or other **aggregation functions**
- Recommend to user C the **most similar** item i by computing similarity in the common feature space
 - **Distributional distance** measures also work well
- Benefit: does not require other users' data (comparing to collaborative filtering)

Limitations of content-based approach

- Finding the appropriate features
 - e.g., images, movies, music
 - Embeddings and deep learning can help
- Overspecialization
 - Never recommends items outside user's content profile
 - People might have multiple interests
- Recommendations for new users
 - How to build a profile?

Collaborative Filtering

- Common insight: personal tastes are correlated
 - If Alice and Bob both like X and Alice likes Y then Bob is more likely to like Y

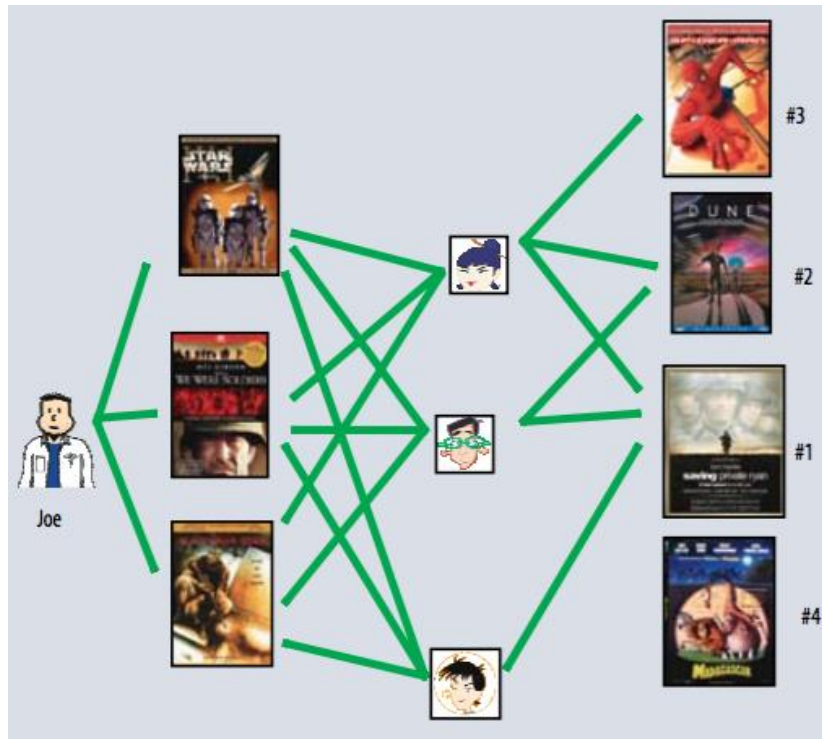
	Harry Potter 1	Harry Potter 2	Harry Potter 3	Twilight	Star Wars 1	Star Wars 2	Star Wars 3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

Two users are similar if they rate the **same items** in a **similar way**

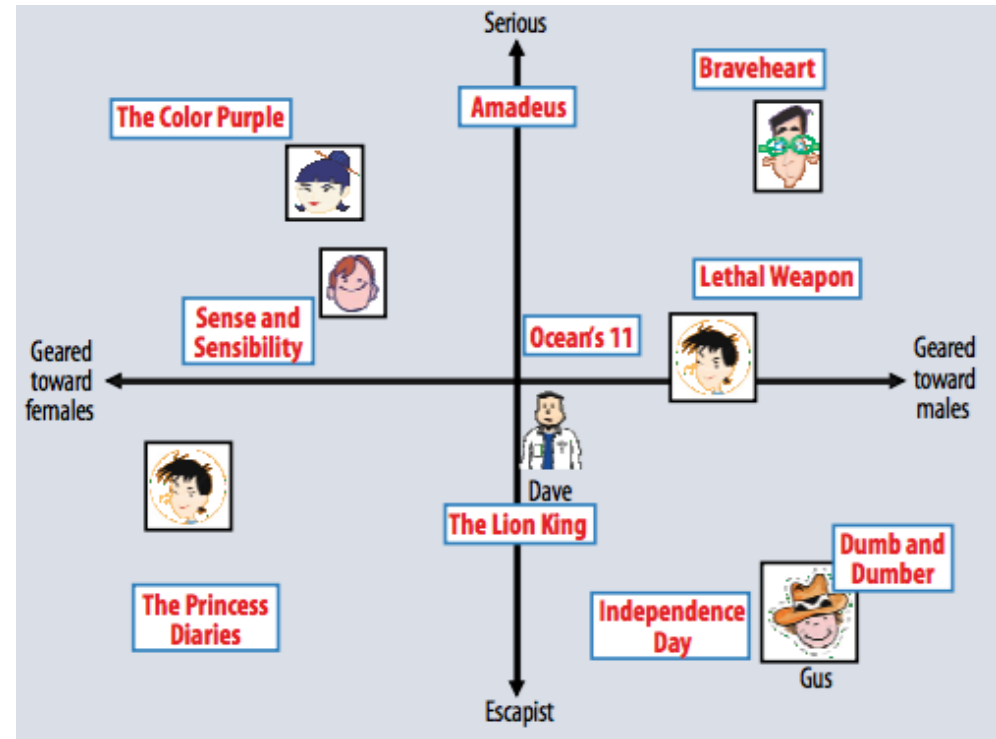
Recommend to user C the items liked by **many** of the **most similar users**.

Two types of collaborative filtering

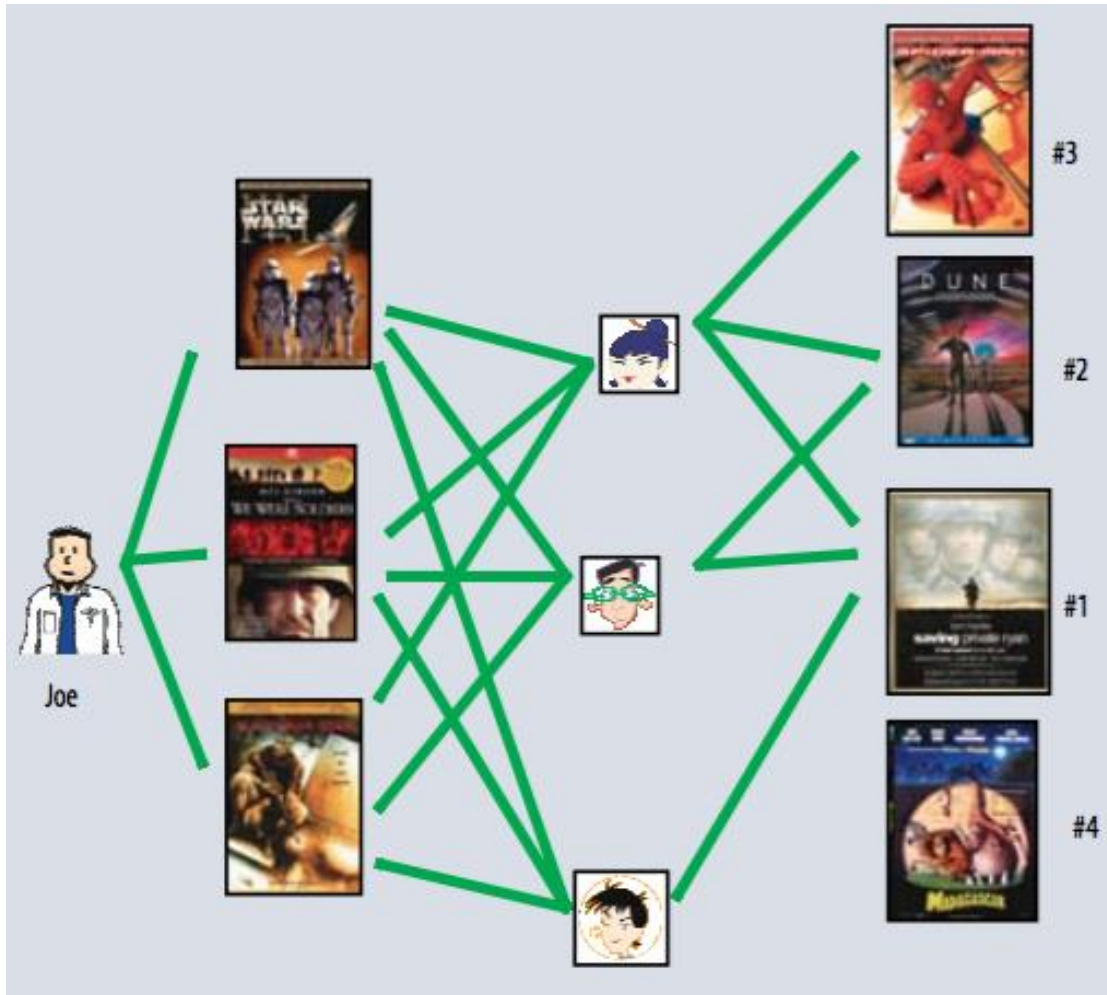
1. Neighborhood Methods



2. Latent Factor Methods



Neighborhood methods



In the figure, assume that a green line indicates the movie was **watched**

Algorithm:

1. Find neighbors based on similarity of movie preferences
2. Recommend movies that those neighbors watched

Neighborhood Methods

User similarity

	Harry Potter 1	Harry Potter 2	Harry Potter 3	Twilight	Star Wars 1	Star Wars 2	Star Wars 3
A	1			1	1		
B	1	1	1				
C				1	1	1	
D		1					1

Jaccard Similarity:

- Convert the ratings (1 indicates reviewed, 0 otherwise)
- Skip dimensions where both users have 0

$$Jsim(A,B) = 1/5$$

$$Jsim(A,C) = 1/2$$

User similarity

	Harry Potter 1	Harry Potter 2	Harry Potter 3	Twilight	Star Wars 1	Star Wars 2	Star Wars 3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

Cosine Similarity:

- Assume zero for missing entries
- Include all dimensions

$$\text{Cos}(A,B) = 0.38$$

$$\text{Cos}(A,C) = 0.32$$

User similarity

	Harry Potter 1	Harry Potter 2	Harry Potter 3	Twilight	Star Wars 1	Star Wars 2	Star Wars 3
A	2/3			5/3	-7/3		
B	1/3	1/3	-2/3				
C				-5/3	1/3	4/3	
D		0					0

Pearson's Correlation:

- Subtract the mean rating per user (without the zeros) and then compute correlation coefficient

$$\text{Corr}(A,B) = 0.092$$

$$\text{Corr}(A,C) = -0.559$$

User-user collaborative filtering

- For a **user** u , find the set $\text{Top-}K(u)$ of the **K other users** whose ratings are most “**similar**” to u ’s ratings
- Estimate u ’s ratings based on ratings of users in $\text{Top-}K$ using some **aggregation function**. For item i :

$$\hat{r}_{ui} = \frac{1}{Z} \sum_{v \in \text{Top}K(u)} \text{sim}(u, v) r_{vi}$$
$$Z = \sum_{v \in \text{Top}K(u)} \text{sim}(u, v)$$

- Modeling **deviations**:

$$\hat{r}_{ui} = \boxed{\bar{r}_u} + \frac{1}{Z} \sum_{v \in \text{Top}K(u)} \text{sim}(u, v) \boxed{(r_{vi} - \bar{r}_v)}$$

Mean rating of u

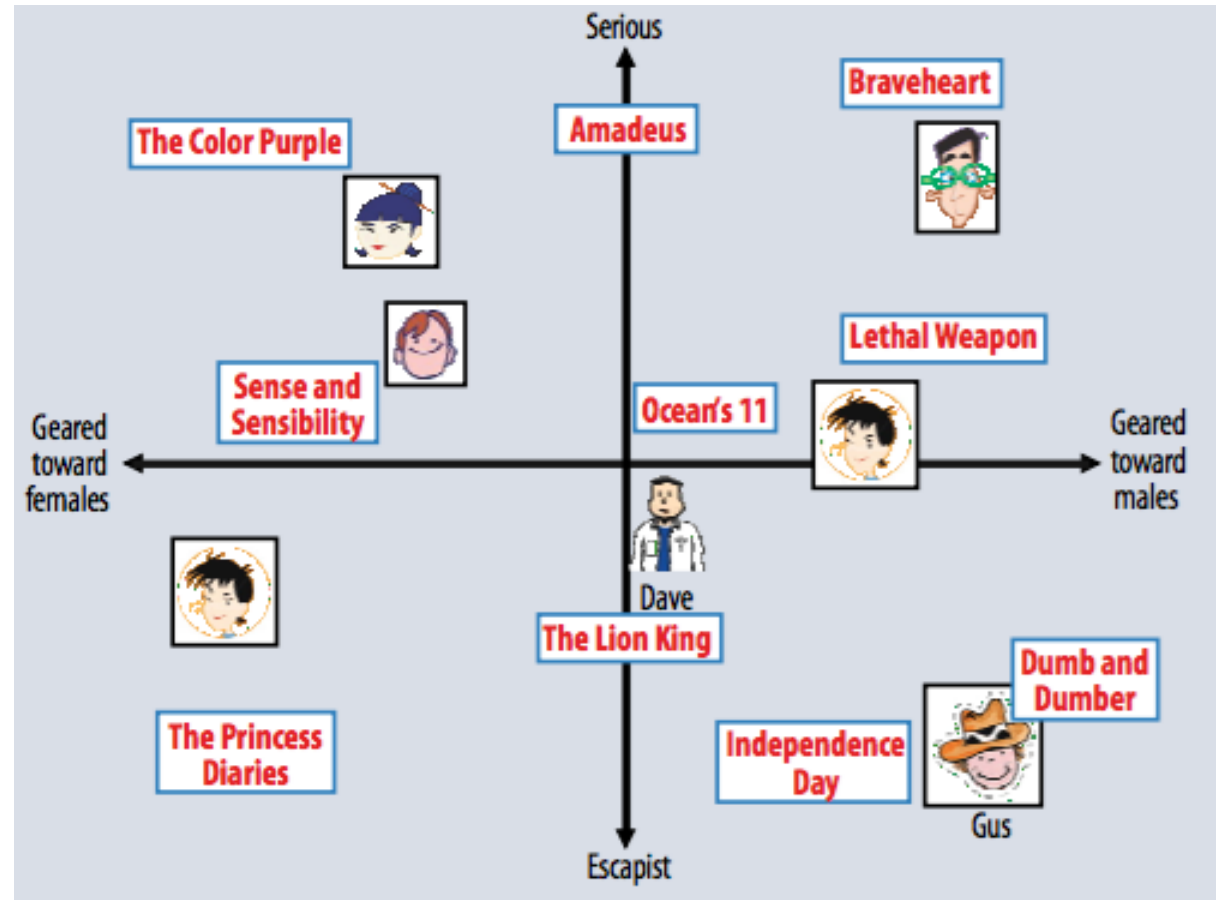
Deviation from mean for v

Mean deviation of similar users

- Advantage: doesn't need the features of items to be given.

Latent factor methods

- Assume that both movies and users live in some **low-dimensional space** describing their properties
- Recommend a movie based on its **proximity** to the user in the latent space



Latent Factor Methods

Matrix factorization

(with vectors)

- User vectors:

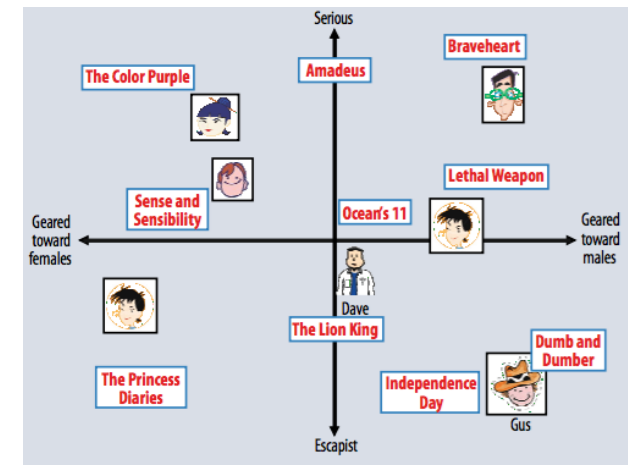
$$\mathbf{w}_u \in \mathbb{R}^r$$

- Item vectors:

$$\mathbf{h}_i \in \mathbb{R}^r$$

- Rating prediction:

$$v_{ui} = \mathbf{w}_u^T \mathbf{h}_i$$



Figures from Koren et al. (2009)

Matrix factorization

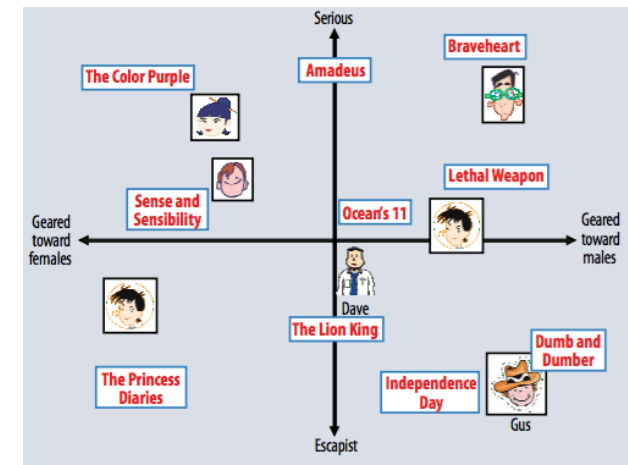
(with vectors)

- Set of non-zero entries:

$$\mathcal{Z} = \{(u, i) : v_{ui} \neq 0\}$$

- Objective:

$$\operatorname{argmin}_{\mathbf{w}, \mathbf{h}} \sum_{(u, i) \in \mathcal{Z}} (v_{ui} - \mathbf{w}_u^T \mathbf{h}_i)^2$$



Figures from Koren et al. (2009)

	Harry Potter 1	Harry Potter 2	Harry Potter 3	Twilight	Star Wars 1	Star Wars 2	Star Wars 3
A	4			5	1		
B	5	5	4				
C				2	4	5	
D		3					3

Matrix factorization

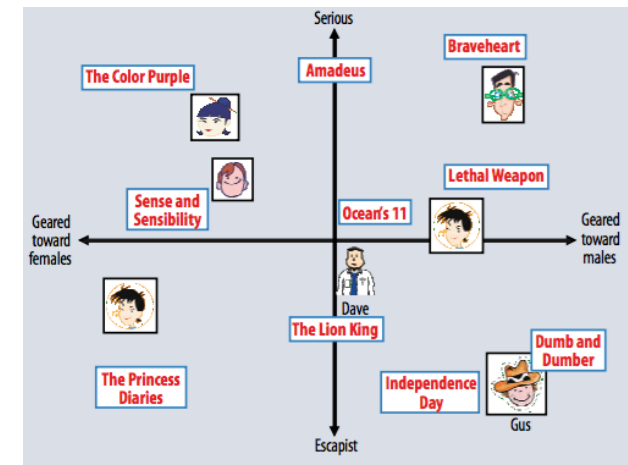
(with vectors)

- Regularized Objective:

$$\begin{aligned} \operatorname{argmin}_{\mathbf{w}, \mathbf{h}} \sum_{(u,i) \in \mathcal{Z}} (v_{ui} - \mathbf{w}_u^T \mathbf{h}_i)^2 \\ + \lambda \left(\sum_i \|\mathbf{w}_i\|^2 + \sum_u \|\mathbf{h}_u\|^2 \right) \end{aligned}$$

- SGD update for random (u,i):

$$\begin{aligned} e_{ui} &\leftarrow v_{ui} - \mathbf{w}_u^T \mathbf{h}_i \\ \mathbf{w}_u &\leftarrow \mathbf{w}_u + \gamma(e_{ui} \mathbf{h}_i - \lambda \mathbf{w}_u) \\ \mathbf{h}_i &\leftarrow \mathbf{h}_i + \gamma(e_{ui} \mathbf{w}_u - \lambda \mathbf{h}_i) \end{aligned}$$



Figures from Koren et al. (2009)

Matrix factorization (with matrices)

- User vectors:

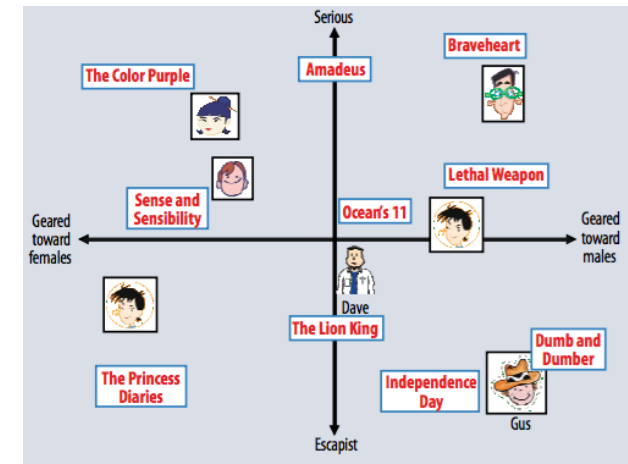
$$(W_{u*})^T \in \mathbb{R}^r$$

- Item vectors:

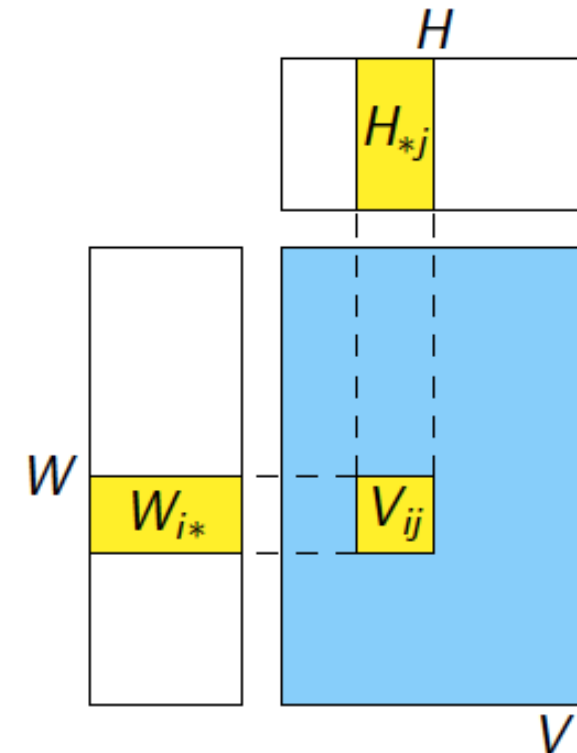
$$H_{*i} \in \mathbb{R}^r$$

- Rating prediction:

$$\begin{aligned} V_{ui} &= W_{u*} H_{*i} \\ &= [WH]_{ui} \end{aligned}$$



Figures from Koren et al. (2009)



Figures from Gemulla et al. (2011)

Matrix factorization

(with matrices)

- SGD

require that the loss can be written as

$$L = \sum_{(i,j) \in Z} l(V_{ij}, W_{i*}, H_{*j})$$

Algorithm 1 SGD for Matrix Factorization

Require: A training set Z , initial values W_0 and H_0

while not converged **do** {step}

 Select a training point $(i, j) \in Z$ uniformly at random.

$W'_{i*} \leftarrow W_{i*} - \epsilon_n N \frac{\partial}{\partial W_{i*}} l(V_{ij}, W_{i*}, H_{*j})$

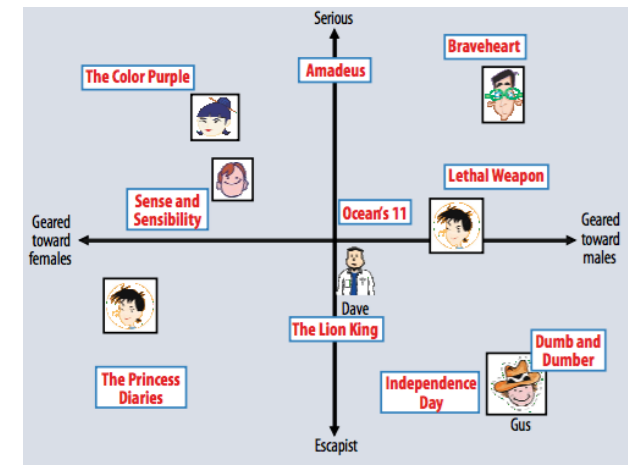
$H_{*j} \leftarrow H_{*j} - \epsilon_n N \frac{\partial}{\partial H_{*j}} l(V_{ij}, W_{i*}, H_{*j})$

$W_{i*} \leftarrow W'_{i*}$

end while

step size

Figure from Gemulla et al. (2011)



Figures from Koren et al. (2009)

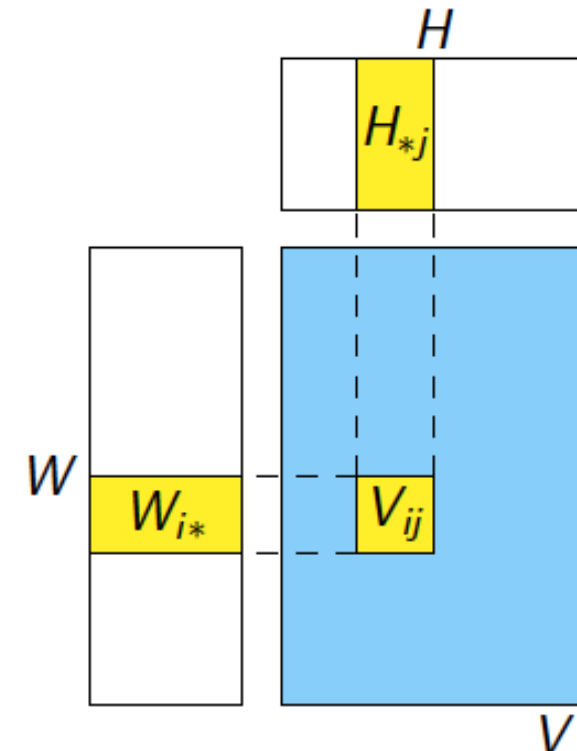


Figure from Gemulla et al. (2011)

Matrix factorization

Example Factors

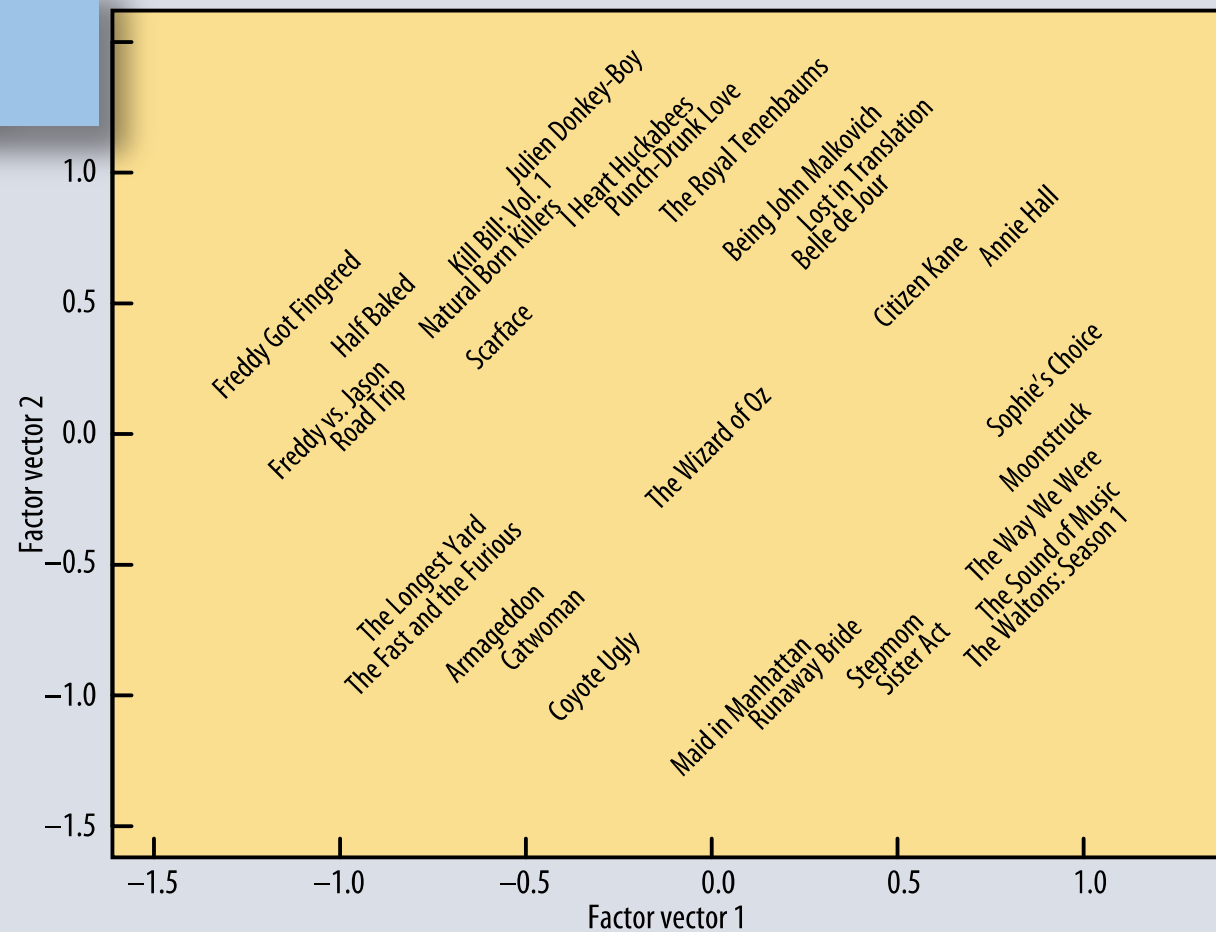


Figure 3. The first two vectors from a matrix decomposition of the Netflix Prize data. Selected movies are placed at the appropriate spot based on their factor vectors in two dimensions. The plot reveals distinct genres, including clusters of movies with strong female leads, fraternity humor, and quirky independent films.

Evaluation

- Split the data into **train** and **test** set
 - Keep a fraction of the ratings to test the accuracy of the predictions
- Metrics:
 - **Root Mean Square Error** (RMSE) for measuring the quality of **predicted ratings**:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i,j} (\widehat{r}_{ij} - r_{ij})^2}$$

- **Precision/Recall** for measuring the quality of **binary (action/no action) predictions**:
 - Precision = fraction of predicted actions that were correct
 - Recall = fraction of actions that were predicted correctly
 - We may also consider the top-k recommendations for the evaluation
- **Kendal' tau** for measuring the quality of predicting the **ranking of items**:
 - The fraction of pairs of items that are ordered correctly (or incorrectly)

Summary

- Recommender systems solve many **real-world** (*large-scale) **problems**
- Collaborative filtering by Matrix Factorization (MF) is an **efficient** and **effective** approach
- MF is just another example of a **common recipe**:
 1. Define a model
 2. Define an objective function
 3. Optimize with SGD

What happened to the Netflix challenge?

- 1M prize to improve the prediction accuracy by 10%

